

ALLOY — Social Job Platform

Database Design Document

Complete Schema | Multilevel Indexes | Django ORM Commands | React + DRF Integration | ERD

Stack: Python / Django / PostgreSQL / React | Version 2.0

1. Platform Overview

Alloy ek social job-discovery platform hai Indian students aur professionals ke liye. Neeche 16 tables hain jo 6 modules mein divide hain. Har table ke saath Django model, SQL commands, aur multilevel indexes diye gaye hain.

Module	Tables	Purpose
Users	users, user_profiles, follows	Auth, profile, social graph
Jobs	jobs_cache, job_categories, saved_jobs	Job listings, categories, wishlist
Preferences	student_profiles, job_preferences, pref_categories	Student-to-job matching
Social Feed	posts, post_media, likes, comments	Home feed & interactions
Messaging	conversations, messages	Direct messaging
Notifications	notifications	All activity alerts

2. Users Module

2.1 users (Est. ~50,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
user_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Unique user ID — BIGINT for 18 trillion scale
username	VARCHAR(50)	NOT NULL UNIQUE	UNIQUE	Login handle, unique across platform
email	VARCHAR(255)	NOT NULL UNIQUE	UNIQUE	Login email, auth & notifications
phone	VARCHAR(15)	UNIQUE NULL	UNIQUE	Optional, OTP login support
password_hash	VARCHAR(255)	NOT NULL	-	bcrypt hash — never plain text
role	ENUM(student,professional,recruiter,admin)	DEFAULT student	-	Role-based access control
is_verified	TINYINT(1)	DEFAULT 0	-	Email/phone verification flag
is_active	TINYINT(1)	DEFAULT 1	-	Soft deactivation — 0=deactivated
created_at	TIMESTAMP	DEFAULT NOW()	-	Account creation time
last_login_at	TIMESTAMP	NULL	-	Last login tracking

Multilevel Indexes — users

Index Name	Columns	Purpose / Query it serves
idx_users_email_active	email, is_active	Login query: WHERE email=? AND is_active=1
idx_users_phone_verified	phone, is_verified	OTP login: WHERE phone=? AND is_verified=1
idx_users_role_created	role, created_at	Admin dashboard: users by role sorted by date

Django Model

```
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    ROLE_CHOICES = [
        ('student', 'Student'), ('professional', 'Professional'),
        ('recruiter', 'Recruiter'), ('admin', 'Admin')]
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='student')
    phone = models.CharField(max_length=15, null=True, blank=True, unique=True)
    is_verified = models.BooleanField(default=False)

    class Meta:
```

```
indexes = [
    models.Index(fields=['email', 'is_active'], name='idx_users_email_active'),
    models.Index(fields=['phone', 'is_verified'], name='idx_users_phone_verified'),
    models.Index(fields=['role', 'created_at'], name='idx_users_role_created'),
]
```

Raw SQL — Create Table + Indexes

```
CREATE TABLE users (
    user_id      BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    username     VARCHAR(50)  NOT NULL UNIQUE,
    email        VARCHAR(255) NOT NULL UNIQUE,
    phone        VARCHAR(15)  UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role         ENUM('student', 'professional', 'recruiter', 'admin') DEFAULT 'student',
    is_verified  TINYINT(1)   DEFAULT 0,
    is_active    TINYINT(1)   DEFAULT 1,
    created_at   TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    last_login_at TIMESTAMP    NULL
);

CREATE INDEX idx_users_email_active  ON users(email, is_active);
CREATE INDEX idx_users_phone_verified ON users(phone, is_verified);
CREATE INDEX idx_users_role_created  ON users(role, created_at);
```

2.2 user_profiles (Est. ~50,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
profile_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Profile primary key
user_id	BIGINT UNSIGNED	NOT NULL UNIQUE	FK -> users.user_id	1:1 with users — UNIQUE enforces one profile per user
full_name	VARCHAR(100)	NOT NULL	-	Display name
bio	TEXT	NULL	-	About-me text
avatar_url	VARCHAR(500)	NULL	-	CDN profile photo URL
city	VARCHAR(100)	NULL	INDEX	Current city — indexed for location matching
state	VARCHAR(100)	NULL	-	State for regional filtering
followers_count	INT UNSIGNED	DEFAULT 0	-	Cached counter — O(1) read
following_count	INT UNSIGNED	DEFAULT 0	-	Cached counter — O(1) read
updated_at	TIMESTAMP	ON UPDATE NOW()	-	Auto-updates on any change

Multilevel Indexes — user_profiles

Index Name	Columns	Purpose / Query it serves
idx_profile_city_state	city, state	Location filter: WHERE city=? AND state=?
idx_profile_user_name	user_id, full_name	Profile lookup + display name in one shot

Django Model

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')
    full_name = models.CharField(max_length=100)
    bio = models.TextField(null=True, blank=True)
    avatar_url = models.URLField(max_length=500, null=True, blank=True)
    city = models.CharField(max_length=100, null=True, blank=True)
    state = models.CharField(max_length=100, null=True, blank=True)
    followers_count = models.PositiveIntegerField(default=0)
    following_count = models.PositiveIntegerField(default=0)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        indexes = [
            models.Index(fields=['city', 'state'], name='idx_profile_city_state'),
            models.Index(fields=['user', 'full_name'], name='idx_profile_user_name'),
        ]
```

Django ORM Usage

```
| # Get profile with user in one query (no N+1)
```

```
profile = UserProfile.objects.select_related('user').get(user_id=uid)

# Find all users in Mumbai
profiles = UserProfile.objects.filter(city='Mumbai', state='Maharashtra')
```

2.3 follows (Est. ~500,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
follower_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Who is following
following_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Who is being followed
created_at	TIMESTAMP	DEFAULT NOW()	-	Follow timestamp

Composite PK	PRIMARY KEY(follower_id, following_id) — prevents duplicates + acts as index. No separate UNIQUE needed.
---------------------	--

Multilevel Indexes — follows

Index Name	Columns	Purpose / Query it serves
PRIMARY KEY	follower_id, following_id	Is A following B? O(1) lookup
idx_follows_following_id	following_id, created_at	Who follows me? Sorted by newest
idx_follows_follower_id	follower_id, created_at	Who am I following? Sorted by newest

Django Model + ORM

```
class Follow(models.Model):
    follower = models.ForeignKey(User, on_delete=models.CASCADE, related_name='following')
    following = models.ForeignKey(User, on_delete=models.CASCADE, related_name='followers')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('follower', 'following') # Composite PK equivalent
        indexes = [
            models.Index(fields=['following', 'created_at'], name='idx_follows_following_id'),
            models.Index(fields=['follower', 'created_at'], name='idx_follows_follower_id'),
        ]

# Usage
# Follow someone
Follow.objects.get_or_create(follower=request.user, following=target_user)

# Get my followers
followers = Follow.objects.filter(following=user).select_related('follower__profile')
```

3. Jobs Module

3.1 job_categories (18 rows — fixed)

Column	Data Type	Constraints	Key / Index	Purpose
category_id	TINYINT UNSIGNED	AUTO_INCREMENT PK	PK	TINYINT — only 18 categories, BIGINT would waste 8x storage
tag	VARCHAR(60)	NOT NULL UNIQUE	UNIQUE	Adzuna tag: e.g. healthcare-nursing-jobs
label	VARCHAR(80)	NOT NULL	-	Display: Healthcare & Nursing
icon	VARCHAR(10)	NULL	-	Emoji icon for UI
sort_order	TINYINT UNSIGNED	DEFAULT 0	-	Display order in filter panel

Django Model

```
class JobCategory(models.Model):
    tag = models.CharField(max_length=60, unique=True)
    label = models.CharField(max_length=80)
    icon = models.CharField(max_length=10, null=True, blank=True)
    sort_order = models.PositiveSmallIntegerField(default=0)

    class Meta:
        ordering = ['sort_order']
        verbose_name_plural = 'Job Categories'
```

3.2 jobs_cache (Est. ~900 rows, refreshed daily)

Column	Data Type	Constraints	Key / Index	Purpose
job_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Internal job ID
adzuna_id	VARCHAR(50)	NOT NULL UNIQUE	UNIQUE	Adzuna original ID — prevents duplicate cache
category_id	TINYINT UNSIGNED	NOT NULL	FK -> job_categories	FK to category — part of multilevel index
title	VARCHAR(255)	NOT NULL	INDEX	Job title — part of FULLTEXT index
company_name	VARCHAR(200)	NOT NULL	INDEX	Company — part of FULLTEXT index
location_city	VARCHAR(100)	NULL	INDEX	City — multilevel index with category
contract_time	ENUM(full_time, part_time)	NULL	INDEX	Schedule type
contract_type	ENUM(permanent, temporary)	NULL	INDEX	Job type

Column	Data Type	Constraints	Key / Index	Purpose
	t,contract)			
salary_min	DECIMAL(12,2)	NULL	INDEX	Min salary — range query index
salary_max	DECIMAL(12,2)	NULL	-	Max salary
description	TEXT	NULL	INDEX	FULLTEXT indexed
redirect_url	VARCHAR(1000)	NOT NULL	-	Apply button URL
is_remote	TINYINT(1)	DEFAULT 0	INDEX	Remote flag — multilevel with category
is_internship	TINYINT(1)	DEFAULT 0	INDEX	Internship flag
posted_at	TIMESTAMP	NOT NULL	INDEX	Job post date — for date filter
cached_at	TIMESTAMP	DEFAULT NOW()	-	Cache insertion time
expires_at	TIMESTAMP	NOT NULL	-	Cache expiry — 24h refresh cycle

Multilevel Indexes — jobs_cache (Most Critical Table)

Index Name	Columns	Purpose / Query it serves
idx_jobs_cat_city_remote	category_id, location_city, is_remote	Main filter: category + city + remote toggle
idx_jobs_cat_contract	category_id, contract_time, contract_type	Category + schedule + type combined filter
idx_jobs_cat_salary	category_id, salary_min, salary_max	Category + salary range query
idx_jobs_cat_posted	category_id, posted_at DESC	Category + newest jobs first
idx_jobs_cat_internship	category_id, is_internship, posted_at	Internship filter sorted by date
idx_jobs_city_contract	location_city, contract_time, is_remote	City-first search with schedule filter
idx_ft_jobs (FULLTEXT)	title, description, company_name	Keyword search — 10x faster than LIKE %

Django Model

```
class JobsCache(models.Model):
    adzuna_id = models.CharField(max_length=50, unique=True)
    category = models.ForeignKey(JobCategory, on_delete=models.RESTRICT,
    related_name='jobs')
    title = models.CharField(max_length=255)
    company_name = models.CharField(max_length=200)
    location_city = models.CharField(max_length=100, null=True, blank=True)
    contract_time = models.CharField(max_length=20, choices=[('full_time', 'Full Time'),
    ('part_time', 'Part Time')], null=True)
    contract_type = models.CharField(max_length=20, choices=[('permanent', 'Permanent'),
    ('contract', 'Contract')], null=True)
```

```

    salary_min = models.DecimalField(max_digits=12, decimal_places=2, null=True)
    salary_max = models.DecimalField(max_digits=12, decimal_places=2, null=True)
    description = models.TextField(null=True, blank=True)
    redirect_url = models.URLField(max_length=1000)
    is_remote = models.BooleanField(default=False)
    is_internship = models.BooleanField(default=False)
    posted_at = models.DateTimeField()
    expires_at = models.DateTimeField()

    class Meta:
        indexes = [
            models.Index(fields=['category', 'location_city', 'is_remote'],
name='idx_jobs_cat_city_remote'),
            models.Index(fields=['category', 'contract_time', 'contract_type'],
name='idx_jobs_cat_contract'),
            models.Index(fields=['category', 'salary_min', 'salary_max'],
name='idx_jobs_cat_salary'),
            models.Index(fields=['category', 'posted_at'],
name='idx_jobs_cat_posted'),
            models.Index(fields=['category', 'is_internship', 'posted_at'],
name='idx_jobs_cat_internship'),
            models.Index(fields=['location_city', 'contract_time', 'is_remote'],
name='idx_jobs_city_contract'),
        ]
        # FULLTEXT index: add via raw SQL migration (Django ORM doesn't support FULLTEXT natively)

```

FULLTEXT Index

Django ORM FULLTEXT support nahi karta — migration file mein manually add karo:
 RunSQL('CREATE FULLTEXT INDEX idx_ft_jobs ON jobs_cache(title, description, company_name)')

Django ORM Usage Examples

```

# Main jobs filter query – uses idx_jobs_cat_city_remote
jobs = JobsCache.objects.filter(
    category_id=3,
    location_city='Mumbai',
    is_remote=True,
    expires_at__gt=timezone.now()
).order_by('-posted_at')

# Salary range filter – uses idx_jobs_cat_salary
jobs = JobsCache.objects.filter(
    category_id=3,
    salary_min__gte=50000,
    salary_max__lte=200000
)

# Internship filter – uses idx_jobs_cat_internship
jobs = JobsCache.objects.filter(category_id=3, is_internship=True).order_by('-posted_at')

# FULLTEXT keyword search (raw SQL required)
from django.db import connection
with connection.cursor() as cursor:
    cursor.execute(
        "SELECT * FROM jobs_cache WHERE MATCH(title,description,company_name) AGAINST(%s IN BOOLEAN MODE)",
        [keyword]
    )

```

|)

3.3 saved_jobs (Est. ~100,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
user_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Who saved — part of composite PK
job_id	BIGINT UNSIGNED	NOT NULL	FK -> jobs_cache.job_id	Which job — part of composite PK
saved_at	TIMESTAMP	DEFAULT NOW()	-	Save timestamp
note	VARCHAR(500)	NULL	-	User's personal note on saved job

PK Design

PRIMARY KEY(user_id, job_id) — composite PK acts as unique constraint + index. ORDER BY saved_at DESC for saved jobs list.

Django Model + ORM

```
class SavedJob(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='saved_jobs')
    job = models.ForeignKey(JobsCache, on_delete=models.CASCADE, related_name='saved_by')
    saved_at = models.DateTimeField(auto_now_add=True)
    note = models.CharField(max_length=500, null=True, blank=True)

    class Meta:
        unique_together = ('user', 'job')
        ordering = ['-saved_at']

# Save a job
SavedJob.objects.get_or_create(user=request.user, job_id=job_id)

# Get all saved jobs with job details
saved = SavedJob.objects.filter(user=request.user).select_related('job__category')
```

4. Student Job Preference System

Core Feature	Student apna academic field enter karta hai → System Adzuna API params automatically build karta hai → Personalized job feed milta hai.
--------------	---

4.1 student_profiles (Est. ~40,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
student_profile_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Student profile PK
user_id	BIGINT UNSIGNED	NOT NULL UNIQUE	FK -> users.user_id	1:1 with users
college_name	VARCHAR(200)	NOT NULL	INDEX	Indexed for college-wise analytics
degree	ENUM(B.Tech,B CA,BBA,MBA,M. Tech,MBBS,PhD, Other)	NOT NULL	-	Degree type
field_of_study	VARCHAR(150)	NOT NULL	INDEX	e.g. Computer Science — key matching field
graduation_year	YEAR	NOT NULL	-	Freshers vs experienced filter
current_year	TINYINT UNSIGNED	NULL	-	1st/2nd/3rd/4th year — internship timing
cgpa	DECIMAL(4,2)	NULL	-	CGPA — some companies filter on this
skills_text	TEXT	NULL	-	Comma-separated: Python, React, SQL
is_actively_looking	TINYINT(1)	DEFAULT 1	-	Job hunting flag

Multilevel Indexes — student_profiles

Index Name	Columns	Purpose / Query it serves
idx_student_field_degree	field_of_study, degree	Match students: CS + B.Tech
idx_student_college_year	college_name, graduation_year	College-year analytics
idx_student_active_field	is_actively_looking, field_of_study	Active job seekers by field

Django Model + ORM

```
class StudentProfile(models.Model):
    DEGREE_CHOICES = [
        ('B.Tech', 'B.Tech'), ('BCA', 'BCA'), ('BBA', 'BBA'),
        ('MBA', 'MBA'), ('M.Tech', 'M.Tech'), ('MBBS', 'MBBS'),
        ('PhD', 'PhD'), ('Other', 'Other')]
    user = models.OneToOneField(User, on_delete=models.CASCADE)
```

```
college_name      = models.CharField(max_length=200)
degree            = models.CharField(max_length=20, choices=DEGREE_CHOICES)
field_of_study    = models.CharField(max_length=150)
graduation_year   = models.PositiveSmallIntegerField()
current_year      = models.PositiveSmallIntegerField(null=True)
cgpa              = models.DecimalField(max_digits=4, decimal_places=2, null=True)
skills_text       = models.TextField(null=True, blank=True)
is_actively_looking= models.BooleanField(default=True)

class Meta:
    indexes = [
        models.Index(fields=['field_of_study', 'degree'],
name='idx_student_field_degree'),
        models.Index(fields=['college_name', 'graduation_year'],
name='idx_student_college_year'),
        models.Index(fields=['is_actively_looking', 'field_of_study'],
name='idx_student_active_field'),
    ]
```

4.2 job_preferences (Est. ~40,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
pref_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Preference PK
user_id	BIGINT UNSIGNED	NOT NULL UNIQUE	FK -> users.user_id	1:1 with users
preferred_role	VARCHAR(200)	NULL	-	→ Adzuna what= param
preferred_city	VARCHAR(100)	NULL	INDEX	→ Adzuna where= param
open_to_remote	TINYINT(1)	DEFAULT 0	-	Include remote jobs flag
open_to_internship	TINYINT(1)	DEFAULT 0	-	Include internships flag
contract_time_pref	ENUM(full_time, part_time, any)	DEFAULT any	-	→ Adzuna contract_time=
contract_type_pref	ENUM(permanent, contract, any)	DEFAULT any	-	→ Adzuna contract_type=
salary_min_expected	DECIMAL(12,2)	NULL	-	→ Adzuna salary_min=
sort_preference	ENUM(date, salary, relevance)	DEFAULT date	-	→ Adzuna sort_by=
max_days_old	TINYINT UNSIGNED	DEFAULT 7	-	→ Adzuna max_days_old=

Multilevel Indexes — job_preferences

Index Name	Columns	Purpose / Query it serves
idx_pref_city_remote	preferred_city, open_to_remote	City-based preference lookup
idx_pref_role_city	preferred_role, preferred_city	Role + city combination filter

Django Model + Adzuna Param Builder

```
class JobPreference(models.Model):
    TIME_CHOICES = [('full_time', 'Full Time'), ('part_time', 'Part Time'), ('any', 'Any')]
    TYPE_CHOICES = [('permanent', 'Permanent'), ('contract', 'Contract'), ('any', 'Any')]
    SORT_CHOICES = [('date', 'Date'), ('salary', 'Salary'), ('relevance', 'Relevance')]

    user = models.OneToOneField(User, on_delete=models.CASCADE)
    preferred_role = models.CharField(max_length=200, null=True, blank=True)
    preferred_city = models.CharField(max_length=100, null=True, blank=True)
    open_to_remote = models.BooleanField(default=False)
    open_to_internship = models.BooleanField(default=False)
    contract_time_pref = models.CharField(max_length=20, choices=TIME_CHOICES, default='any')
    salary_min_expected = models.DecimalField(max_digits=12, decimal_places=2, null=True)
    sort_preference = models.CharField(max_length=20, choices=SORT_CHOICES, default='date')
    max_days_old = models.PositiveSmallIntegerField(default=7)

    def build_adzuna_params(self):
        params = {'results_per_page': 50, 'sort_by': self.sort_preference}
        if self.preferred_role:
            params['what'] = self.preferred_role
```

```

        if self.preferred_city: params['where'] = self.preferred_city
        if self.contract_time_pref != 'any': params['contract_time'] = self.contract_time_pref
        if self.salary_min_expected: params['salary_min'] = float(self.salary_min_expected)
        params['max_days_old'] = self.max_days_old
        return params

    class Meta:
        indexes = [
            models.Index(fields=['preferred_city', 'open_to_remote'],
name='idx_pref_city_remote'),
            models.Index(fields=['preferred_role', 'preferred_city'],
name='idx_pref_role_city'),
        ]

```

4.3 pref_categories (Est. ~80,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
pref_id	BIGINT UNSIGNED	NOT NULL	FK -> job_preferences.pref_id	FK to student preference
category_id	TINYINT UNSIGNED	NOT NULL	FK -> job_categories.category_id	FK to Adzuna category
priority	TINYINT UNSIGNED	DEFAULT 1	-	1=top choice → API call order
added_at	TIMESTAMP	DEFAULT NOW()	-	When category was added

Bridge Table

M:N between job_preferences and job_categories. Student multiple categories prefer kar sakta hai. priority field determines which category is fetched first from Adzuna.

Django Model + ORM

```

class PrefCategory(models.Model):
    preference = models.ForeignKey(JobPreference, on_delete=models.CASCADE,
related_name='categories')
    category = models.ForeignKey(JobCategory, on_delete=models.RESTRICT,
related_name='preferred_by')
    priority = models.PositiveSmallIntegerField(default=1)
    added_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('preference', 'category')
        ordering = ['priority']

# Get student's Adzuna params + top category in ONE query
pref = JobPreference.objects.select_related('user').prefetch_related(
    'categories__category'
).get(user_id=uid)

top_category = pref.categories.first().category.tag # priority=1
params = pref.build_adzuna_params()

```

```
| params['category'] = top_category
```

5. Social Feed Module

5.1 posts (Est. ~500,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
post_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Post PK
user_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Post author
content	TEXT	NULL	-	Post text — NULL for image-only posts
post_type	ENUM(text,image,video,job_share)	DEFAULT text	INDEX	Type filter
shared_job_id	BIGINT UNSIGNED	NULL	FK -> jobs_cache.job_id	FK only for job_share type
likes_count	INT UNSIGNED	DEFAULT 0	-	Cached counter — trigger-updated
comments_count	INT UNSIGNED	DEFAULT 0	-	Cached counter — trigger-updated
is_deleted	TINYINT(1)	DEFAULT 0	-	Soft delete — data retained for audit
created_at	TIMESTAMP	DEFAULT NOW()	INDEX	Feed sort key

Multilevel Indexes — posts

Index Name	Columns	Purpose / Query it serves
idx_posts_user_deleted_date	user_id, is_deleted, created_at DESC	My posts: only active, newest first
idx_posts_type_date	post_type, is_deleted, created_at DESC	Filter by type: job_share posts in feed
idx_posts_job_type	shared_job_id, post_type	All posts sharing a specific job

Django Model + ORM

```
class Post(models.Model):
    TYPE_CHOICES = [('text', 'Text'), ('image', 'Image'), ('video', 'Video'), ('job_share', 'Job Share')]
    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, related_name='posts')
    content = models.TextField(null=True, blank=True)
    post_type = models.CharField(max_length=20, choices=TYPE_CHOICES, default='text')
    shared_job = models.ForeignKey(JobsCache, on_delete=models.SET_NULL, null=True, blank=True)
    likes_count = models.PositiveIntegerField(default=0)
    comments_count = models.PositiveIntegerField(default=0)
```

```

is_deleted = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    indexes = [
        models.Index(fields=['user', 'is_deleted', 'created_at'],
name='idx_posts_user_deleted_date'),
        models.Index(fields=['post_type', 'is_deleted', 'created_at'],
name='idx_posts_type_date'),
        models.Index(fields=['shared_job', 'post_type'],
name='idx_posts_job_type'),
    ]

# Home feed – following users' posts
following_ids = Follow.objects.filter(follower=request.user).values_list('following_id',
flat=True)
feed = Post.objects.filter(
    user_id__in=following_ids, is_deleted=False
).select_related('user__profile').prefetch_related('media').order_by('-created_at')[:20]

```

5.2 likes (Est. ~5,000,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
user_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Who liked — part of composite PK
post_id	BIGINT UNSIGNED	NOT NULL	FK -> posts.post_id	Which post — part of composite PK
created_at	TIMESTAMP	DEFAULT NOW()	-	Like timestamp

Cached Counter Pattern

INSERT INTO likes → Trigger: UPDATE posts SET likes_count = likes_count + 1. O(1) read instead of COUNT(*) on 5M rows = 100x faster.

Django Model + ORM

```

class Like(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    post = models.ForeignKey(Post, on_delete=models.CASCADE, related_name='likes')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('user', 'post') # Composite PK – no duplicate likes
        indexes = [
            models.Index(fields=['post', 'created_at'], name='idx_likes_post_date'),
        ]

# Like a post (with counter update in one transaction)
from django.db import transaction
with transaction.atomic():
    like, created = Like.objects.get_or_create(user=request.user, post_id=post_id)
    if created:
        Post.objects.filter(pk=post_id).update(likes_count=models.F('likes_count')+1)

```

5.3 comments (Est. ~2,000,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
comment_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Comment PK
post_id	BIGINT UNSIGNED	NOT NULL	FK -> posts.post_id	Parent post
user_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Comment author
parent_comment_id	BIGINT UNSIGNED	NULL	FK -> comments.comment_id	NULL=top-level, non-NULL=reply (self-join)
content	TEXT	NOT NULL	-	Comment text
is_deleted	TINYINT(1)	DEFAULT 0	-	Soft delete
created_at	TIMESTAMP	DEFAULT NOW()	-	Comment time

Multilevel Indexes — comments

Index Name	Columns	Purpose / Query it serves
idx_comments_post_parent_date	post_id, parent_comment_id, created_at	Post comments: top-level first, then replies
idx_comments_user_date	user_id, is_deleted, created_at	User's comment history

Django Model + ORM

```
class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE,
related_name='comments')
    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
    parent_comment = models.ForeignKey('self', on_delete=models.SET_NULL, null=True,
blank=True, related_name='replies')
    content = models.TextField()
    is_deleted = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['post', 'parent_comment', 'created_at'],
name='idx_comments_post_parent_date'),
            models.Index(fields=['user', 'is_deleted', 'created_at'],
name='idx_comments_user_date'),
        ]

# Get top-level comments + replies
top_comments = Comment.objects.filter(
    post_id=post_id, parent_comment=None, is_deleted=False
).select_related('user__profile').prefetch_related(
```

```
models.Prefetch('replies',  
queryset=Comment.objects.filter(is_deleted=False).select_related('user__profile'))  
) .order_by('created_at')
```

6. Messaging & Notifications Module

6.1 conversations (Est. ~200,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
conversation_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Conversation PK
user1_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Always the LOWER user_id (canonical ordering)
user2_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Always the HIGHER user_id
last_message_at	TIMESTAMP	NULL	INDEX	Indexed for inbox sort
created_at	TIMESTAMP	DEFAULT NOW()	-	Conversation start time

Canonical Order

ALWAYS store user1_id < user2_id. UNIQUE(user1_id, user2_id) prevents duplicate conversations. Lookup: WHERE (user1_id=A AND user2_id=B) OR (user1_id=B AND user2_id=A) → simplified to canonical form.

Multilevel Indexes — conversations

Index Name	Columns	Purpose / Query it serves
UNIQUE(user1_id, user2_id)	user1_id, user2_id	Canonical pair lookup — prevent duplicate conversations
idx_conv_user1_last_msg	user1_id, last_message_at DESC	User1's inbox sorted by recent messages
idx_conv_user2_last_msg	user2_id, last_message_at DESC	User2's inbox sorted by recent messages

Django Model + ORM

```
class Conversation(models.Model):
    user1 = models.ForeignKey(User, on_delete=models.RESTRICT,
related_name='conversations_as_user1')
    user2 = models.ForeignKey(User, on_delete=models.RESTRICT,
related_name='conversations_as_user2')
    last_message_at= models.DateTimeField(null=True)
    created_at      = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('user1', 'user2')
        indexes = [
            models.Index(fields=['user1', 'last_message_at'], name='idx_conv_user1_last_msg'),
            models.Index(fields=['user2', 'last_message_at'], name='idx_conv_user2_last_msg'),
        ]

    @classmethod
    def get_or_create_conversation(cls, user_a, user_b):
        # Always store lower ID as user1
```

```

u1, u2 = (user_a, user_b) if user_a.id < user_b.id else (user_b, user_a)
conv, _ = cls.objects.get_or_create(user1=u1, user2=u2)
return conv

```

6.2 messages (Est. ~10,000,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
message_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Message PK
conversation_id	BIGINT UNSIGNED	NOT NULL	FK -> conversations.conversation_id	Parent conversation
sender_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Message sender
content	TEXT	NOT NULL	-	Message text
is_read	TINYINT(1)	DEFAULT 0	INDEX	Unread badge — multilevel index
is_deleted	TINYINT(1)	DEFAULT 0	-	Soft delete
created_at	TIMESTAMP	DEFAULT NOW()	INDEX	Chat scroll order

Multilevel Indexes — messages

Index Name	Columns	Purpose / Query it serves
idx_msg_conv_date	conversation_id, created_at DESC	Chat scroll: last 50 messages instantly
idx_msg_conv_unread	conversation_id, sender_id, is_read	Unread count per conversation per sender
idx_msg_sender_date	sender_id, is_deleted, created_at	User's sent message history

Django Model + ORM

```

class Message(models.Model):
    conversation = models.ForeignKey(Conversation, on_delete=models.CASCADE,
related_name='messages')
    sender = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='sent_messages')
    content = models.TextField()
    is_read = models.BooleanField(default=False)
    is_deleted = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['conversation', 'created_at'],
name='idx_msg_conv_date'),
            models.Index(fields=['conversation', 'sender', 'is_read'],
name='idx_msg_conv_unread'),
            models.Index(fields=['sender', 'is_deleted', 'created_at'],
name='idx_msg_sender_date'),

```

```

    ]

    # Get last 50 messages in a chat
    msgs = Message.objects.filter(conversation_id=conv_id, is_deleted=False)
        .select_related('sender__profile').order_by('-created_at')[:50]

    # Mark messages as read
    Message.objects.filter(conversation_id=conv_id, is_read=False)
        .exclude(sender=request.user).update(is_read=True)

```

6.3 notifications (Est. ~20,000,000 rows)

Column	Data Type	Constraints	Key / Index	Purpose
notif_id	BIGINT UNSIGNED	AUTO_INCREMENT PK	PK	Notification PK
user_id	BIGINT UNSIGNED	NOT NULL	FK -> users.user_id	Recipient — multilevel index
actor_id	BIGINT UNSIGNED	NULL	FK -> users.user_id	Who triggered — NULL for system notifs
type	ENUM(like, comment, follow, job_match, mention)	NOT NULL	INDEX	Notification type
ref_post_id	BIGINT UNSIGNED	NULL	-	Related post (like/comment)
ref_job_id	BIGINT UNSIGNED	NULL	-	Related job (job_match)
is_read	TINYINT(1)	DEFAULT 0	INDEX	Unread badge count
created_at	TIMESTAMP	DEFAULT NOW()	INDEX	Sort key

Multilevel Indexes — notifications

Index Name	Columns	Purpose / Query it serves
idx_notif_user_read_date	user_id, is_read, created_at DESC	Inbox: unread first, sorted by newest
idx_notif_user_type_date	user_id, type, created_at DESC	Filter by type: only likes, only follows
idx_notif_actor_date	actor_id, type, created_at	Activity log: what has this user triggered

Django Model + ORM

```

class Notification(models.Model):
    TYPE_CHOICES = [('like', 'Like'), ('comment', 'Comment'), ('follow', 'Follow'),
                    ('job_match', 'Job Match'), ('mention', 'Mention')]
    user = models.ForeignKey(User, on_delete=models.CASCADE,
                             related_name='notifications')
    actor = models.ForeignKey(User, on_delete=models.CASCADE, null=True,
                              related_name='triggered_notifs')

```

```
type = models.CharField(max_length=20, choices=TYPE_CHOICES)
ref_post = models.ForeignKey(Post, on_delete=models.CASCADE, null=True, blank=True)
ref_job = models.ForeignKey(JobsCache, on_delete=models.CASCADE, null=True, blank=True)
is_read = models.BooleanField(default=False)
created_at = models.DateTimeField(auto_now_add=True)

class Meta:
    indexes = [
        models.Index(fields=['user', 'is_read', 'created_at'],
name='idx_notif_user_read_date'),
        models.Index(fields=['user', 'type', 'created_at'],
name='idx_notif_user_type_date'),
        models.Index(fields=['actor', 'type', 'created_at'],
name='idx_notif_actor_date'),
    ]

# Unread count badge (super fast – uses idx_notif_user_read_date)
unread_count = Notification.objects.filter(user=request.user, is_read=False).count()
```

7. Django REST Framework + React Integration

Stack

Backend: Django + DRF (djangorestframework) + SimpleJWT | Frontend: React + Axios + Context API

7.1 DRF Setup (settings.py)

```
# Install: pip install djangorestframework djangorestframework-simplejwt django-cors-headers

INSTALLED_APPS = [
    ...existing apps...
    'rest_framework',
    'corsheaders',
    'users', 'jobs', 'feed', 'messaging', 'notifications',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # Must be FIRST
    ...rest of middleware...
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES':
    ['rest_framework_simplejwt.authentication.JWTAuthentication'],
    'DEFAULT_PERMISSION_CLASSES': ['rest_framework.permissions.IsAuthenticated'],
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20
}

CORS_ALLOWED_ORIGINS = ['http://localhost:3000'] # React dev server
CORS_ALLOW_CREDENTIALS = True

from datetime import timedelta
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
}
```

7.2 Serializers Example (jobs/serializers.py)

```
from rest_framework import serializers
from .models import JobsCache, JobCategory

class JobCategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = JobCategory
        fields = ['category_id', 'tag', 'label', 'icon']

class JobSerializer(serializers.ModelSerializer):
    category = JobCategorySerializer(read_only=True)

    class Meta:
        model = JobsCache
```

```
fields = ['job_id', 'title', 'company_name', 'location_city', 'contract_time',
          'contract_type', 'salary_min', 'salary_max', 'is_remote',
          'is_internship', 'posted_at', 'redirect_url', 'category']
```

7.3 ViewSets + URL Routing (jobs/views.py + urls.py)

```
# jobs/views.py
from rest_framework import viewsets, filters
from django_filters.rest_framework import DjangoFilterBackend
from .models import JobsCache
from .serializers import JobSerializer

class JobViewSet(viewsets.ReadOnlyModelViewSet):
    serializer_class = JobSerializer
    filter_backends = [DjangoFilterBackend, filters.OrderingFilter]
    filterset_fields = ['category', 'location_city', 'is_remote', 'is_internship',
                       'contract_time', 'contract_type']
    ordering_fields = ['posted_at', 'salary_min']

    def get_queryset(self):
        return JobsCache.objects.filter(
            expires_at__gt=tz.now()
        ).select_related('category').order_by('-posted_at')

# jobs/urls.py
from rest_framework.routers import DefaultRouter
from .views import JobViewSet

router = DefaultRouter()
router.register(r'jobs', JobViewSet, basename='jobs')
urlpatterns = router.urls

# project/urls.py
urlpatterns = [
    path('api/', include('jobs.urls')),
    path('api/auth/', include('rest_framework_simplejwt.urls')),
]
```

7.4 React — Axios Setup + API Service

```
// src/services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000/api/',
  headers: { 'Content-Type': 'application/json' }
});

// Auto-attach JWT token to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

```
// Auto-refresh token on 401
api.interceptors.response.use(
  res => res,
  async err => {
    if (err.response?.status === 401) {
      const refresh = localStorage.getItem('refresh_token');
      const { data } = await axios.post('http://localhost:8000/api/auth/token/refresh/',
{ refresh });
      localStorage.setItem('access_token', data.access);
      err.config.headers.Authorization = `Bearer ${data.access}`;
      return axios(err.config); // Retry original request
    }
    return Promise.reject(err);
  }
);

// API calls
export const jobsAPI = {
  getJobs: (params) => api.get('jobs/', { params }),
  saveJob: (jobId) => api.post(`saved-jobs/`, { job_id: jobId }),
  unsaveJob: (jobId) => api.delete(`saved-jobs/${jobId}/`),
};

export const feedAPI = {
  getFeed: (page) => api.get(`posts/?page=${page}`),
  likePost: (id) => api.post(`posts/${id}/like/`,
  addComment: (id,content) => api.post(`posts/${id}/comments/`, { content }),
};

export const authAPI = {
  login: (email,pwd) => api.post('auth/token/', { username:email, password:pwd }),
  register: (data) => api.post('auth/register/', data),
};

export default api;
```

7.5 React — Jobs Component Example

```
// src/components/Jobs.jsx
import { useState, useEffect } from 'react';
import { jobsAPI } from '../services/api';

export default function Jobs() {
  const [jobs, setJobs] = useState([]);
  const [loading, setLoading] = useState(false);
  const [filters, setFilters] = useState({
    category: '', location_city: '', is_remote: false, is_internship: false
  });

  useEffect(() => {
    fetchJobs();
  }, [filters]);

  const fetchJobs = async () => {
    setLoading(true);
```

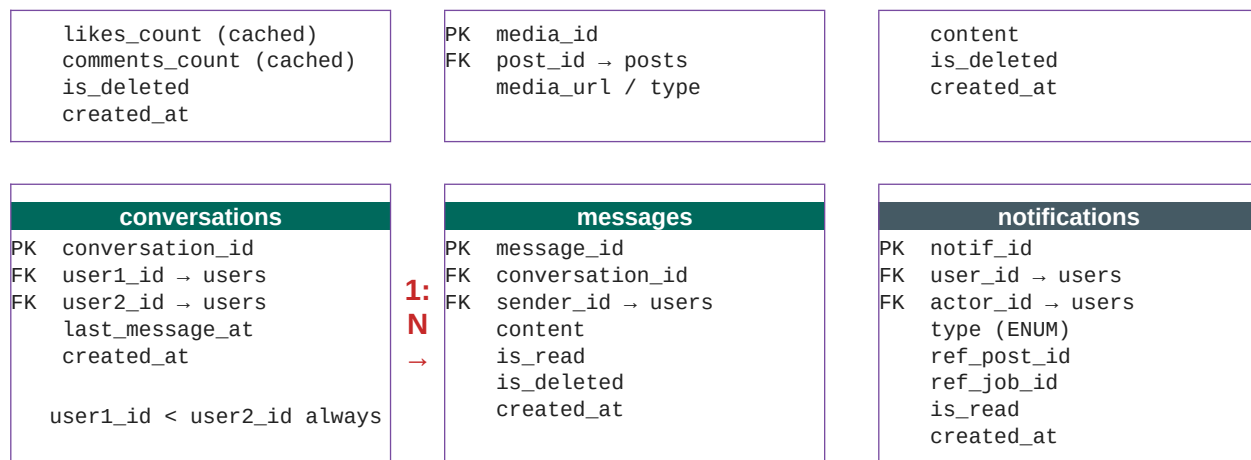
```
    try {
      const { data } = await jobsAPI.getJobs(filters);
      setJobs(data.results);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
      {loading && <p>Loading...</p>}
      {jobs.map(job => (
        <div key={job.job_id}>
          <h3>{job.title}</h3>
          <p>{job.company_name} | {job.location_city}</p>
          <a href={job.redirect_url} target='_blank'>Apply</a>
        </div>
      ))}
    </div>
  );
}
```

8. Complete Relationship Map

Parent Table	Type	Child Table	ON DELETE	Reason
users	1:1	user_profiles	CASCADE	Orphan profile nahi chahiye
users	1:1	student_profiles	CASCADE	Student data without user = useless
users	1:1	job_preferences	CASCADE	Dead preferences = stale API calls
users	N:N	follows	CASCADE	User delete → follow links cleanup
users	1:N	posts	SET NULL	Soft delete — content moderation
posts	1:N	post_media	CASCADE	Post delete → media CDN cleanup
users	N:N	likes	CASCADE	User/post delete → likes delete
users	1:N	comments	SET NULL	Soft delete — thread preserved
users	N:N	saved_jobs	CASCADE	User delete → saved list cleanup
job_categories	1:N	jobs_cache	RESTRICT	Category delete blocked if jobs exist
job_preferences	1:N	pref_categories	CASCADE	Pref delete → categories delete
job_categories	1:N	pref_categories	RESTRICT	Category delete blocked if preferred
conversations	1:N	messages	CASCADE	Conversation delete → all messages delete
users	1:N	notifications	CASCADE	User delete → notifications cleanup

9. Entity Relationship Diagram (ERD)

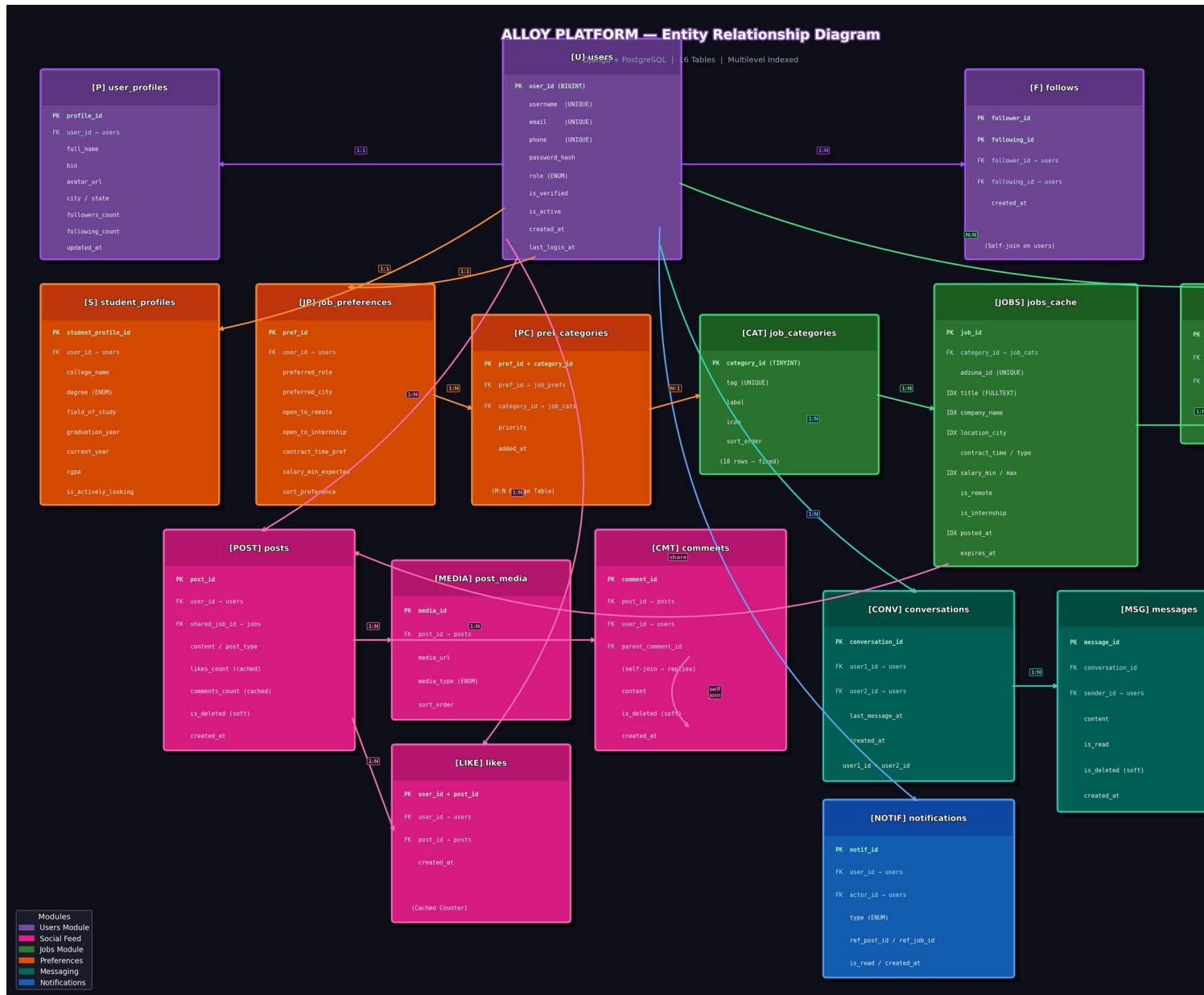


ERD Legend

PK = Primary Key	FK = Foreign Key	UQ = Unique	1:1 = OneToOne	1:N = OneToMany	N:N = M2M Bridge
------------------	------------------	-------------	----------------	-----------------	------------------

ERD — Visual Relationship Map

Color coded by module. Arrows show FK relationships. Arrow label = relationship type (1:1, 1:N, N:N).



Module Color Legend

Users Module	Social Feed	Jobs Module	Preferences	Messaging	Notifications
--------------	-------------	-------------	-------------	-----------	---------------